In [1]:
```
# importing major libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
df=pd.read_csv('Employee_promotion.csv').iloc[:,-4:]
df
```

Out[2]:

	JobTitle	Education	City	Promotion
0	Software Engineer	Bachelor's	Atlanta	No Promotion
1	Full Stack Developer	Master's	Milwaukee	No Promotion
2	Web Developer	Bachelor's	Las Vegas	No Promotion
3	BI Developer	Master's	Minneapolis	No Promotion
4	BI Developer	PhD	Austin	No Promotion
...	...	...	...	...
1195	SRE	Bachelor's	Jacksonville	No Promotion
1196	ML Engineer	Bachelor's	Nashville	Got Promotion
1197	Data Engineer	Bachelor's	Milwaukee	No Promotion
1198	BI Developer	Bachelor's	San Antonio	No Promotion
1199	Data Scientist	Master's	Chicago	No Promotion

1200 rows × 4 columns

In [3]:
```
# Label encoder
```

In [4]:
```
#manual  way
```

In [5]:
```
df.Promotion.unique()
#encoding
df.Promotion.map({'No Promotion':0,'Got Promotion':1})
```

Out[5]:
```
0       0
1       0
2       0
3       0
4       0
       ..
1195    0
1196    1
1197    0
1198    0
1199    0
Name: Promotion, Length: 1200, dtype: int64
```

In [6]:
```
#scikit learn
#label encoder

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

x=df.iloc[:,0:-1]
y=df.Promotion
```

In [7]:
```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_stat
```

In [8]:
```
print('rows for training',x_train.shape[0])
print('rows for testing',x_test.shape[0])
```

```
rows for training 960
rows for testing 240
```

In [9]:
```
le = LabelEncoder() # 1D array #Series #y_train,y_test
le
```

Out[9]:
```
LabelEncoder()
```

In [10]:
```
#fitting
le.fit(y_train)
```

Out[10]:
```
LabelEncoder()
```

In [11]:
```
#transform
y_train_le = le.transform(y_train)
y_test_le = le.transform(y_test)
```

In [12]:
```
y_test_le
```

Out[12]:
```
array([1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1,
       1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1,
       0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1,
       0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1,
       0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1,
       1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0])
```

In [13]:
```
# le.inverse_transform (y_test_le)
```

In [14]:
```
# one hot encoding
# 2nd arrays
# dataframes
# norminal variables
```

In [15]:
```
# manual way
```

In [16]:
```
df.JobTitle.unique()
```

Out[16]:
```
array(['Software Engineer', 'Full Stack Developer', 'Web Developer',
       'BI Developer', 'Business Analyst', 'Data Analyst',
       'Data Engineer', 'Product Manager', 'Data Scientist',
       'DevOps Engineer', 'Research Scientist', 'ML Engineer',
       'Quality Engineer', 'SRE', 'Database Administrator'], dtype=object)
```

In [17]:
```
pd.get_dummies(df.JobTitle,dtype=int)
```

Out[17]:

	BI Developer	Business Analyst	Data Analyst	Data Engineer	Data Scientist	Database Administrator	DevOp Enginee
0	0	0	0	0	0	0	
1	0	0	0	0	0	0	
2	0	0	0	0	0	0	
3	1	0	0	0	0	0	
4	1	0	0	0	0	0	
...	...	...	...	...	...	...	
1195	0	0	0	0	0	0	
1196	0	0	0	0	0	0	
1197	0	0	0	1	0	0	
1198	1	0	0	0	0	0	
1199	0	0	0	0	1	0	

1200 rows × 15 columns

In [18]:
```
# sciket Learn
# OneHotEncoder

from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False)
ohe
```

Out[18]:
```
▾ OneHotEncoder
OneHotEncoder(sparse_output=False)
```

In [19]:
```
# dataframe
```

In [20]:
```
ohe.fit(x_train[['JobTitle']])
```

Out[20]:

OneHotEncoder

OneHotEncoder(sparse_output=False)

In [21]:
```
pd.DataFrame(ohe.transform(x_train[['JobTitle']]),columns=ohe.get_feature_name
```

Out[21]:

	JobTitle_BI Developer	JobTitle_Business Analyst	JobTitle_Data Analyst	JobTitle_Data Engineer	JobTitle_Data Scientist	J
0	0.0	0.0	0.0	1.0	0.0	
1	0.0	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	1.0	0.0	
...	...	...	...	...	...	
955	0.0	0.0	0.0	0.0	0.0	
956	0.0	0.0	1.0	0.0	0.0	
957	0.0	1.0	0.0	0.0	0.0	
958	0.0	0.0	0.0	0.0	0.0	
959	0.0	0.0	0.0	0.0	0.0	

960 rows × 15 columns

In [22]:
```
ohe.get_feature_names_out()
```

Out[22]:
```
array(['JobTitle_BI Developer', 'JobTitle_Business Analyst',
       'JobTitle_Data Analyst', 'JobTitle_Data Engineer',
       'JobTitle_Data Scientist', 'JobTitle_Database Administrator',
       'JobTitle_DevOps Engineer', 'JobTitle_Full Stack Developer',
       'JobTitle_ML Engineer', 'JobTitle_Product Manager',
       'JobTitle_Quality Engineer', 'JobTitle_Research Scientist',
       'JobTitle_SRE', 'JobTitle_Software Engineer',
       'JobTitle_Web Developer'], dtype=object)
```

In [23]:
```
# on ID
l1 = ['No Promotion','Got Promotion']
ohe_on1d = OneHotEncoder(sparse_output=False,categories=[l1],dtype=int,drop='f
ohe_on1d
```

Out[23]:

```
OneHotEncoder(categories=[['No Promotion', 'Got Promotion']], drop='first',
              dtype=<class 'int'>, sparse_output=False)
```

In [24]:
```
#fit
ohe_on1d.fit(y_train.values.reshape(len(y_train),1))
```

Out[24]:

```
OneHotEncoder(categories=[['No Promotion', 'Got Promotion']], drop='first',
              dtype=<class 'int'>, sparse_output=False)
```

In [25]:
```
y_train_ohe_temp = ohe_on1d.transform(y_train.values.reshape(len(y_train),1))
#ohe_on1d.transform(y_train.values.reshape(len(y_train),1))
```

In [26]:
```
pd.DataFrame(y_train_ohe_temp,columns=ohe_on1d.get_feature_names_out())
```

Out[26]:

	x0_Got Promotion
0	0
1	0
2	0
3	1
4	0
...	...
955	0
956	0
957	0
958	0
959	0

960 rows × 1 columns

In [27]:
```
# ordinal encoding
# manual way
```

In [28]:
```
df.Education.unique()
```

Out[28]:
```
array(["Bachelor's", "Master's", 'PhD', 'High School'], dtype=object)
```

In [29]:
```
df.Education.map({"Bachelor's":1,"Master's":2,'PhD':3,'High School':0}).head(4
```

Out[29]:
```
0    1
1    2
2    1
3    2
Name: Education, dtype: int64
```

In [30]:
```
#scikit learn
#Ordinalencoder
#2d arrays
#dataframes
#ordinal variables

from sklearn.preprocessing import OrdinalEncoder
l1 = ['High School',"Bachelor's","Master's",'PhD']
oe = OrdinalEncoder(categories=[l1],dtype=int)
oe
```

Out[30]:

▾ OrdinalEncoder

```
OrdinalEncoder(categories=[['High School', "Bachelor's", "Master's",
'PhD']],
               dtype=<class 'int'>)
```

In [31]:
```
#fit
oe.fit(x_train[['Education']])
```

Out[31]:

▾ OrdinalEncoder

```
OrdinalEncoder(categories=[['High School', "Bachelor's", "Master's",
'PhD']],
               dtype=<class 'int'>)
```

In [32]:
```
x_train_education_oe=oe.transform(x_train[['Education']])
x_test_education_oe=oe.transform(x_test[['Education']])
```

In [33]:
```
#frequency encoding
#manual way
```

In [34]:
```
df.City.unique()
```

Out[34]:
```
array(['Atlanta', 'Milwaukee', 'Las Vegas', 'Minneapolis', 'Austin',
       'Philadelphia', 'Nashville', 'Chicago', 'Los Angeles', 'Arlington',
       'Kansas City', 'Fort Worth', 'Columbus', 'Oklahoma City',
       'Long Beach', 'Cleveland', 'Riverside', 'Charlotte', 'Detroit',
       'Virginia Beach', 'Washington', 'New Orleans', 'Wichita',
       'San Antonio', 'Phoenix', 'Memphis', 'Seattle', 'Louisville',
       'El Paso', 'Sacramento', 'Baltimore', 'Aurora', 'Corpus Christi',
       'Jacksonville', 'Honolulu', 'San Diego', 'Raleigh', 'Anaheim',
       'Dallas', 'Fresno', 'Lexington', 'Stockton', 'Oakland',
       'Santa Ana', 'Indianapolis', 'Albuquerque', 'San Francisco',
       'Omaha', 'Tucson', 'Tampa', 'Portland', 'New York', 'Bakersfield',
       'St. Paul', 'Denver', 'San Jose', 'Colorado Springs', 'Tulsa',
       'Boston', 'Miami', 'Mesa', 'Houston'], dtype=object)
```

In [35]:
```
temp = df.City.value_counts().to_dict()
df.City.map(temp)
```

Out[35]:
```
0       22
1       15
2       21
3       20
4       19
        ..
1195    12
1196    26
1197    15
1198    25
1199    25
Name: City, Length: 1200, dtype: int64
```

In [36]:
```
# target encoding

# new column add on
df['flagged_promotion'] = df.Promotion.map({'No Promotion':0, 'Got Promotion':
df.sample(5)
```

Out[36]:

	JobTitle	Education	City	Promotion	flagged_promotion
1169	DevOps Engineer	Bachelor's	Las Vegas	Got Promotion	1
880	Data Scientist	PhD	Fresno	Got Promotion	1
54	Web Developer	Master's	Atlanta	No Promotion	0
903	Software Engineer	Bachelor's	Houston	No Promotion	0
838	Data Engineer	PhD	Charlotte	No Promotion	0

In [37]:
```
temp = df.groupby('City')['flagged_promotion'].mean()
temp.sample(5)
```

Out[37]:
```
City
Los Angeles    0.066667
Denver         0.250000
Tulsa          0.200000
Mesa           0.071429
Honolulu       0.200000
Name: flagged_promotion, dtype: float64
```

In [38]:
```
df.City.map(temp)
```

Out[38]:
```
0       0.227273
1       0.266667
2       0.238095
3       0.150000
4       0.105263
          ...
1195    0.083333
1196    0.153846
1197    0.266667
1198    0.240000
1199    0.080000
Name: City, Length: 1200, dtype: float64
```

In [ ]:
```

```